

Document Rotation Pipeline

A clean Python + OpenCV mini project for learning pipeline structure, file handling, logging, and multithreading

1. Project Overview

This project builds a small but realistic document-processing pipeline. The idea is to take all images from an input folder, read each image safely, decide what orientation it should be in, rotate it if needed, and save the final image in an output folder. Along the way, we add clean code structure, logging, error handling, and multithreading so the project behaves like a real internship-style pipeline.

- Input: a folder containing images.
- Processing: read, resize, detect orientation, rotate, and save.
- Output: corrected images in a separate folder.
- Extras: logs, folder automation, clean modular code, and threading.

2. Why This Project Is Useful

This is a great practice project because it teaches the same habits used in production AI and computer vision work.

Instead of writing one giant script, you learn to split logic into modules. That makes your code easier to read, test, debug, and extend.

- File handling and folder automation.
- Clean coding with separate functions and files.
- Image preprocessing with OpenCV.
- Logging and error handling for reliability.
- Multithreading for faster batch image processing.

3. Pipeline Flow

```
graph TD; A[Input Folder] --> B[Read Images]; B --> C[Resize / Preprocess]; C --> D[Detect Orientation]; D --> E[Rotate Correctly]; E --> F[Save Output];
```

Input Folder

↓

Read Images

↓

Resize / Preprocess

↓

Detect Orientation

↓

Rotate Correctly

↓

Save Output

↓

Write Logs

Each step has one responsibility. That idea is called separation of concerns.

4. Folder Structure

```
document_pipeline/  
|  
├─ input/  
├─ output/  
├─ logs/  
|  
├─ config.py  
├─ logger.py  
├─ utils.py  
├─ image_processor.py  
└─ main.py
```

5. config.py - central settings

This file stores project constants such as input folder, output folder, log folder, image resize values, and number of worker threads.

Keeping these values in one place makes the project easier to manage. If you want to change the image size or folder name later, you only edit this file.

```
INPUT_DIR = "input"  
OUTPUT_DIR = "output"  
LOG_DIR = "logs"  
  
MAX_WORKERS = 4  
RESIZE_WIDTH = 800  
RESIZE_HEIGHT = 800
```

6. logger.py - track what happens

Logging is important because it helps you understand what the pipeline did for each file.

If an image fails, the log file helps you find the exact reason without guessing.

```
import logging  
import os  
from config import LOG_DIR
```

```

os.makedirs(LOG_DIR, exist_ok=True)

log_file = os.path.join(LOG_DIR, "pipeline.log")

logging.basicConfig(
    filename=log_file,
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logger = logging.getLogger()

```

7. utils.py - helper functions

This file contains small reusable functions that do general tasks like creating folders and collecting image paths.

Keeping helpers separate prevents repetition and keeps main logic clean.

```

import os

def create_folder(folder_path):
    os.makedirs(folder_path, exist_ok=True)

def get_image_paths(folder_path):
    valid_extensions = [".jpg", ".jpeg", ".png"]
    image_paths = []

    for file in os.listdir(folder_path):
        file_path = os.path.join(folder_path, file)

        if os.path.isfile(file_path):
            ext = os.path.splitext(file)[1].lower()
            if ext in valid_extensions:
                image_paths.append(file_path)

```

```
return image_paths
```

8. image_processor.py - core image logic

This is the heart of the pipeline. It contains image reading, resizing, orientation detection, rotation, and saving.

In a real project, the orientation detection part could later be replaced with a model, OCR logic, or some other intelligent method.

```
import cv2
import os
from logger import logger
from config import OUTPUT_DIR, RESIZE_HEIGHT, RESIZE_WIDTH

def resize_image(image):
    return cv2.resize(image, (RESIZE_WIDTH, RESIZE_HEIGHT))

def detect_orientation(filename):
    filename = filename.lower()

    if "90" in filename:
        return 90
    elif "180" in filename:
        return 180
    elif "270" in filename:
        return 270

    return 0

def rotate_image(image, angle):
    if angle == 90:
        return cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE)
    elif angle == 180:
        return cv2.rotate(image, cv2.ROTATE_180)
    elif angle == 270:
```

```

        return cv2.rotate(image, cv2.ROTATE_90_COUNTERCLOCKWISE)

    return image

def process_image(image_path):
    try:
        filename = os.path.basename(image_path)
        logger.info(f"Processing started: {filename}")

        image = cv2.imread(image_path)
        if image is None:
            logger.error(f"Failed to read image: {filename}")
            return

        image = resize_image(image)
        angle = detect_orientation(filename)
        rotated_image = rotate_image(image, angle)

        output_path = os.path.join(OUTPUT_DIR, filename)
        cv2.imwrite(output_path, rotated_image)

        logger.info(f"Successfully processed: {filename} | Rotation: {angle}")
        print(f"Processed: {filename}")

    except Exception as e:
        logger.error(f"Error processing {image_path}: {e}")

```

9. main.py - pipeline controller

The main file starts the pipeline. It creates folders, collects images, and uses multithreading to process multiple images at the same time.

This file should stay simple. It should coordinate the workflow, not contain every detail.

```

from concurrent.futures import ThreadPoolExecutor
from utils import create_folder, get_image_paths
from image_processor import process_image
from config import INPUT_DIR, OUTPUT_DIR, MAX_WORKERS

```

```

from logger import logger

def main():
    logger.info("Pipeline Started")

    create_folder(OUTPUT_DIR)
    image_paths = get_image_paths(INPUT_DIR)

    if len(image_paths) == 0:
        print("No images found.")
        logger.warning("No images found in input folder.")
        return

    print(f"Total Images Found: {len(image_paths)}")

    with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
        executor.map(process_image, image_paths)

    logger.info("Pipeline Completed")
    print("
Pipeline Completed Successfully")

if __name__ == "__main__":
    main()

```

10. How To Run The Project

Create and activate your conda environment first, then install the required package:

```

conda create -n doc_pipeline python=3.10
conda activate doc_pipeline
pip install opencv-python numpy

```

Put your input images inside the input folder and run:

```
python main.py
```

The processed images will appear in the output folder, and the log file will be created in the logs folder.

11. What You Learn From This Project

- How a small pipeline is structured in real projects.
- How to keep code clean and modular.
- How to handle missing files and bad inputs safely.
- How to use OpenCV in a batch-processing workflow.
- How to speed up work with multithreading.
- How logging helps in debugging and maintenance.

Note: the orientation detection logic is intentionally simple here so you can focus on pipeline structure first. Later, you can replace it with a model-based or OCR-based orientation detector.